

Experiment – 2

Neighbours of a Pixel

AIM: To find the 4-neighbors, 8-neighbors, and diagonal neighbors of a given pixel in an image/matrix.

Theory

In digital image processing, the relationship between pixels is defined based on their spatial adjacency. Pixel connectivity plays an important role in image analysis tasks such as region growing, segmentation, and boundary detection. Connectivity determines how pixels are considered neighbors in a digital image. For a pixel located at position (x,y), three commonly used neighbourhood relationships are defined:

1. 4-point Connectivity

In 4-point connectivity, a pixel is connected to its immediate horizontal and vertical neighbors. These neighbors share a common edge with the reference pixel. The 4-point neighbors are:

$$(x-1,y), (x+1,y), (x,y-1), (x,y+1)$$

2. Diagonal Connectivity

Diagonal connectivity considers only the pixels that touch the reference pixel at the corners. These neighbors share a common vertex with the pixel. The diagonal neighbors are:

$$(x-1,y-1), (x-1,y+1), (x+1,y-1), (x+1,y+1)$$

3. Histogram Equalization

8-point connectivity is the combination of both 4-point and diagonal connectivity. It includes all pixels surrounding the reference pixel and represents the most complete form of neighborhood relationship.

In this experiment, neighborhood relationships are determined using relative coordinate offsets and validated using boundary conditions to ensure that the neighboring pixels lie within the image matrix.

Code

```
def find_neighbors(matrix, target):
    rows, cols = len(matrix), len(matrix[0])
    tr, tc = target
    four = []
    diagonal = []
    eight = []
    for dr in [-1, 0, 1]:
        for dc in [-1, 0, 1]:
            if dr == 0 and dc == 0:
                continue
            nr, nc = tr + dr, tc + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                neighbor = (nr, nc)
                eight.append(neighbor)
                if abs(dr) + abs(dc) == 1:
                    four.append(neighbor)
```

```

        elif abs(dr) == 1 and abs(dc) == 1:
            diagonal.append(neighbor)
    return four, diagonal, sorted(eight)

def visualize(matrix, target, four, diagonal):
    print("\nVisualization:")
    print("T = Target | F = 4-point | D = Diagonal | others = value\n")
    for r in range(len(matrix)):
        for c in range(len(matrix[0])):
            if (r, c) == target:
                print("T ", end="")
            elif (r, c) in four:
                print("F ", end="")
            elif (r, c) in diagonal:
                print("D ", end="")
            else:
                print(f"{matrix[r][c]} ", end="")
        print()

def main():
    rows = int(input("Enter number of rows: "))
    cols = int(input("Enter number of columns: "))
    print("\nEnter matrix rows (space-separated):")
    matrix = []
    for i in range(rows):
        row = list(map(int, input(f"Row {i}: ").split()))
        if len(row) != cols:
            print("Invalid row length. Exiting.")
            return
        matrix.append(row)
    print("\nMatrix:")
    for row in matrix:
        print(row)
    tr = int(input("\nEnter target row: "))
    tc = int(input("Enter target column: "))
    if not (0 <= tr < rows and 0 <= tc < cols):
        print("Target pixel out of bounds.")
        return
    target = (tr, tc)
    four, diagonal, eight = find_neighbors(matrix, target)
    print(f"\n4-point neighbors: {four}")
    print(f"Diagonal neighbors: {diagonal}")
    print(f"8-point neighbors: {eight}")
    visualize(matrix, target, four, diagonal)

if __name__ == "__main__":
    main()

```

Output

```
(riptide) PS D:\RAJ ARYAN\Clg\third year\dip\lab> python adjacency.py
Enter number of rows: 6
Enter number of columns: 6

Enter matrix rows (space-separated):
Row 0: 1 2 3 4 5 6
Row 1: 9 8 7 6 5 4
Row 2: 2 4 2 0 4 9
Row 3: 0 3 9 3 5 6
Row 4: 8 9 3 2 0 4
Row 5: 1 2 0 3 9 7

Matrix:
[1, 2, 3, 4, 5, 6]
[9, 8, 7, 6, 5, 4]
[2, 4, 2, 0, 4, 9]
[0, 3, 9, 3, 5, 6]
[8, 9, 3, 2, 0, 4]
[1, 2, 0, 3, 9, 7]

Enter target row: 3
Enter target column: 4

4-point neighbors: [(2, 4), (3, 3), (3, 5), (4, 4)]
Diagonal neighbors: [(2, 3), (2, 5), (4, 3), (4, 5)]

Visualization:
T = Target | F = 4-point | D = Diagonal | others = value

1 2 3 4 5 6
9 8 7 6 5 4
2 4 2 D F D
0 3 9 F T F
8 9 3 D F D
1 2 0 3 9 7
```

Results and Discussion

The program successfully identified the 4-point, diagonal, and 8-point neighbors of a selected target pixel within a given matrix. The 4-point neighbors correctly represented horizontal and vertical adjacency, while diagonal neighbors represented corner adjacency. The 8-point neighbors included all valid surrounding pixels of the target. The visualization output clearly distinguished the target pixel, its 4-point neighbors, and diagonal neighbors, making the spatial relationships easy to understand. The results confirm that loop-based offset checking is an efficient and systematic approach for determining pixel connectivity. This experiment validates the theoretical concepts of pixel adjacency and connectivity used in digital image processing.